

R Como una calculadora

Tipos de datos

Mario A. Morales Rivera

`mamorales@correo.unicordoba.edu.co`

Universidad de Córdoba



Contenido I

- 1 Tipos de datos en R
- 2 Usando R como una calculadora
- 3 Secuencias
- 4 Funciones
- 5 Mensajes de error
- 6 Redondeo
- 7 Caracteres



Tipos de datos en R



Tipos de datos en R



Numéricos:

Numeric: Números reales, pueden tener decimales (ej: 3.14, -2.5).

```
a=3.14  
class(a)
```

```
[1] "numeric"
```

Integer: Números enteros (ej: 5, -10).

```
b1=5L  
class(b1)
```

```
[1] "integer"
```

```
b2=-10L  
class(b2)
```

```
[1] "integer"
```



```
b1=5 # si no se especifica como 5L lo toma como real  
class(b1)
```

```
[1] "numeric"
```



La diferencia está en la memoria usada

```
# Creamos un vector de 1 millón de enteros
vector_enteros <- rep(5L, 1000000)

# Creamos un vector de 1 millón de numéricos (reales)
vector_numericos <- rep(5, 1000000)

c("Entero"=object.size(vector_enteros),"Real"=object.size(vector_numericos))
```

Entero	Real
4000048	8000048



Complex: Números complejos (ej: $3+2i$).

```
c1= 3+2i  
class(c1)
```

```
[1] "complex"
```



Caracteres:

Character: Texto, siempre encerrado entre comillas (ej: "Trat 1", "Análisis de datos").

```
ch="Trat 1"  
class(ch)
```

```
[1] "character"
```



Lógicos:

Logical: Valores booleanos, TRUE o FALSE.

```
l=TRUE  
class(l)
```

```
[1] "logical"
```



Funciones is.

Se usan para indagar el tipo de dato que es un objeto

```
is.numeric(b2)
```

```
[1] TRUE
```

```
is.logical(ch)
```

```
[1] FALSE
```

! Importante

Observe que estas funciones regresan un valor lógico (logical)



Tipos de datos básicos en R II

Factores:

Factor: Categorías o niveles de una variable, utilizados para variables cualitativas (ej: "Hombre", "Mujer").



Estructuras de Datos

Vectores:

Secuencia de elementos del **mismo tipo** (numéricos, caracteres, lógicos).

Matrices:

Arreglos bidimensionales de *elementos* del mismo tipo.

Data frames:

Estructura de datos similar a una hoja de cálculo, con columnas de (posiblemete) diferentes tipos.



Estructuras de Datos (II)

Listas:

Objetos que pueden contener diferentes tipos de elementos, incluyendo otros objetos.



Conocer el tipo de dato

```
1 class(3.5)
2 class(3)
3 class(1:3)
4 class(2L)
```

```
1 class(3+2i)
2 class('Hola mundo')
3 class("Hola mundo")
4 class(TRUE)
5 class(FALSE)
6 class(NA)
```



Funciones is

Estas funciones regresan TRUE si el objeto es del tipo indicado después del punto y FALSE en otro caso.

```
is.numeric(3)
is.integer(3)
is.integer(3L)
is.numeric("M")
is.character("M")
is.logical(TRUE)
is.logical(NA)
is.logical("M")
```



Aplicación

Código que toma un objeto x, si es numérico eleva al cuadrado, en caso contrario manda un mensaje de error.

```
1 x="M"  
2 #x=4  
3 if(is.numeric(x)){  
4   x^2  
5 }else{ cat("Se necesita un numérico")  
6 }
```

Se necesita un numérico



Usando R como una calculadora



R como una calculadora

Qué aprenderemos:

- Poder usar R como calculadora científica
- Poder asignar una variable y ver su valor



Operaciones

```
2+2 # suma
```

```
[1] 4
```

```
5-3 # resta
```

```
[1] 2
```

```
3*5 # producto
```

```
[1] 15
```



Potencia

```
2^2 # potencia
```

```
[1] 4
```

```
9^(1/2)
```

```
[1] 3
```

```
### también se puede usar **
```

```
2**2
```

```
[1] 4
```

```
9**(1/2)
```

```
[1] 3
```



Cociente

6/3

[1] 2



Residuo de la división

```
6%%2
```

```
[1] 0
```

```
10%%4
```

```
[1] 2
```



parte entera de la división

```
6 % / %4
```

```
[1] 1
```



Ejemplo de aplicación

El siguiente código genera un vector x numérico de longitud n , escribir el código R que indique si el número n es par o es impar.

Ayuda: La longitud de un vector x se obtiene con `length(x)`

```
x=round(rnorm(sample(10:50,1)))
```

La longitud de x es: 35

```
n=length(x)
if(n%%2==0){cat("n es par")}
  }else{cat("n es impar")}
```

n es impar



Varias operaciones juntas

$$4-2*3^2$$

$$[1] \quad -14$$



Orden de las operaciones

```
# el orden de las operaciones se altera con parentices  
(4-2)*3^2
```

```
[1] 18
```



Tres formas de asignar valores a un objeto

- La primera forma es usando <-

```
x<-10
```

```
x
```

```
[1] 10
```

- La segunda forma es usando =

```
y=2
```

```
y
```

```
[1] 2
```

- La tercera forma es usando ->

```
10->x
```



Ejemplo: Valor presente

Suponga que queremos hacer un programa para estimar el valor presente de \$100 recibidos en dos años con una tasa de descuento anual del 8 %. La fórmula relacionada se muestra a continuación

$$Pv = \frac{Fv}{(1 + R)^n}$$

Fv=100

R=0.08

n=2



Ejemplo: Valor presente (cont)

$$Fv / (1+R)^2$$

[1] 85.73388

$$Fv / (1+R)^2 \rightarrow Pv$$

Pv

[1] 85.73388



Vectores

Para asignar un conjunto de valores a un vector, usamos $c(n_1, n_2, \dots, n_k)$, c es una abreviatura de concatenar concatenate.

```
x<-c(1,2.6,4.3,5.25)
```

Para asignar un conjunto de enteros consecutivos secuencia regular, podríamos usar $n_1:n_2$, como $1:10$

```
y<-1:50
```

```
y
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 ...  
[26] 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 ...
```



Vectores (cont.)

```
## desde 5 hasta 1 disminuyendo de 1 en 1  
x<-5:1  
x<-rev(1:5)
```

R distingue entre mayúsculas y minúsculas, lo que significa que la X mayúscula y la x minúscula son variables diferentes.

```
x=10  
X
```



Funciones `ls()` and `rm()`

A veces necesitamos verificar todas las variables existentes (objetos) en la memoria. para eso usamos la función `ls()`.

```
ls()
```

```
[1] "a"  "b1" "b2" "c1" "ch" "Fv" "l"  "n"  "Pv" "R"  "x"  "y"
```

Cuando ya no se necesita una variable, podemos eliminarla de la memoria con la función `rm()`

```
rm(z)
```

```
ls()
```

```
[1] "a"  "b1" "b2" "c1" "ch" "Fv" "l"  "n"  "Pv" "R"  "x"  "y"
```



Borrar más de una

```
# borrar mas de una  
#rm(y,y_g,ycs,yg)  
ls()
```

```
[1] "a"  "b1" "b2" "c1" "ch" "Fv" "l"  "n"  "Pv" "R"  "x"  "y"
```

Para eliminar todas las variables (objetos), tenemos el siguiente código

```
rm(list=ls())  
ls()
```

```
character(0)
```



Imprimir el contenido de un objeto.

Para imprimir una variable de carácter (es decir, una cadena) en la pantalla, podríamos usar las funciones `cat()` o `print()`. Recuerde encerrar las cadenas entre comillas simples o dobles.

```
cat('Hola R!!!!')
```

```
Hola R!!!!
```

```
print('Hola R!!!!')
```

```
[1] "Hola R!!!!"
```



Imprimir el contenido de un objeto.

```
cat('Hola R!!!! \n \n \n') # \n es para nueva linea
```

```
Hola R!!!!
```

```
print('Hola R!!!!')
```

```
[1] "Hola R!!!!"
```

```
print('Hola R!!!! \n \n \n ')
```

```
[1] "Hola R!!!! \n \n \n "
```

```
print('Hola R!!!!') # \n es para nueva linea
```

```
[1] "Hola R!!!!"
```



Secuencias



La función seq()

La función seq() se utiliza para generar un conjunto de valores con cierto patrón.

```
seq(1,19,by=2)
```

```
[1] 1 3 5 7 9 11 13 15 17 19
```

```
x<-seq(1, 11, by = pi)
```

```
x
```

```
[1] 1.000000 4.141593 7.283185 10.424778
```

El comando completo tiene el siguiente formato.

```
seq(from=1,to=3, by=0.5)
```

```
[1] 1.0 1.5 2.0 2.5 3.0
```



La función rep()

La función rep() se usa para repetir el mismo valor n veces, como se muestra a continuación.

```
rep(5,10)
```

```
[1] 5 5 5 5 5 5 5 5 5 5
```

```
rep(c(1,2,3),5)
```

```
[1] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
```

```
rep(c(1,2,3),each=5)
```

```
[1] 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3
```



La función length()

Para encontrar el número de valores (observaciones) que tiene un vector, aplicamos la función `length()`, como se muestra a continuación.

```
length(x)
```

```
[1] 4
```

```
length(rep(5,7))
```

```
[1] 7
```



Enfoque de posición

Hay dos formas de entregar datos a una función: por **posición** y por **palabra clave**. Para la función `seq()`, su formato es `seq(from,to,by)`. En el siguiente código de una línea, usamos la forma de posición. En otras palabras, el valor del argumento de entrada depende de su posición en el conjunto de valores de entrada entregados.

```
x<-seq(1,3,0.5)
```



Enfoque de palabra clave

Para el enfoque de **palabra clave**, agregamos una palabra clave delante de cada valor de entrada, como `from=1`. Una ventaja del enfoque de palabra clave es que el orden de los valores de entrada no influye; Por lo tanto, las siguientes tres líneas son equivalentes.

```
seq(from=1,to=3,by=0.5) # son equivalentes
```

```
[1] 1.0 1.5 2.0 2.5 3.0
```

```
seq(to=3,from=1,by=0.5)
```

```
[1] 1.0 1.5 2.0 2.5 3.0
```

```
seq(by=0.5,to=3,from=1)
```

```
[1] 1.0 1.5 2.0 2.5 3.0
```

Entrada de datos con scan

Otra manera fácil de ingresar datos desde el teclado es usar la función `scan()`



Residuo de la división

El operador `%%` regresa el residuo de la división

```
# residuo de la división
```

```
7%%2
```

```
[1] 1
```

```
# ¿qué valor regresa la operación?
```

```
9%%3
```

```
[1] 0
```



Parte entera de la división

El operador `%/ %` regresa la parte entera de la división

```
7 %/ %2
```

```
[1] 3
```



Funciones



Raíz cuadrada

Calcular la raíz cuadrada de 5, es decir, $\sqrt{5}$

```
sqrt(5)
```

```
[1] 2.236068
```

```
sqrt(9)
```

```
[1] 3
```

```
sqrt(-4)
```

```
[1] NaN
```

```
5/0
```

```
[1] Inf
```



Funciones trigonométricas

Calcular $\sin(\pi/2) \cdot \sin(\pi) \cdot \sin(2 \times \pi)$

```
pi
```

```
[1] 3.141593
```

```
sin(pi/2)
```

```
[1] 1
```

```
sin(pi)
```

```
[1] 1.224647e-16
```

```
sin(2*pi)
```

```
[1] -2.449294e-16
```

```
pi
```

```
[1] 3.141593
```



Funciones trigonométricas: ejercicios

Calcular $\cos(\pi/2) * \cos(\pi) * \cos(2 \times \pi) * \tan(\pi/4)$

```
cos(pi/2)
```

```
[1] 6.123234e-17
```

```
cos(pi)
```

```
[1] -1
```

```
cos(2*pi)
```

```
[1] 1
```

```
tan(pi/4)
```

```
[1] 1
```



Funciones exponencial y logaritmo

Aprenderemos a calcular expresiones de la forma a^x , $a \neq 0$, conocida como función exponencial de base a . También expresiones de la forma $\log_a(x)$, conocida como función logarítmica de base $a \neq 0$, el valor que regresa la función corresponde al exponente al que hay que elevar la base a para obtener x . En otras palabras, $\log_a(x) = y$ si, y solo si, $a^y = x$.



Ejercicios

- 3^4
- e^4 (e es el número de euler, la base de los logarimos naturales)
- $\ln(2)$ logaritmo en base euler de 2 (número al que hay que elevar e para obtener 2.
- $\log_{10}(100)$, es decir el logaritmo en base 10 de 100 o lo que es lo mismo: número al que hay que elevar 10 para obtener 100.
- $\log_2(8)$, número al que hay que elevar 2 para obtener 8.



Solución

```
3^4
```

```
[1] 81
```

```
3**4
```

```
[1] 81
```

```
exp(4)
```

```
[1] 54.59815
```

```
log(2)
```

```
[1] 0.6931472
```

```
log(100, base=10)
```

```
[1] 2
```

```
log10(100)
```



Solución (cont)

```
log(100,base=10)
```

```
[1] 2
```

```
log10(100)
```

```
[1] 2
```

```
log2(8)
```

```
[1] 3
```

```
log(8,base=2)
```

```
[1] 3
```

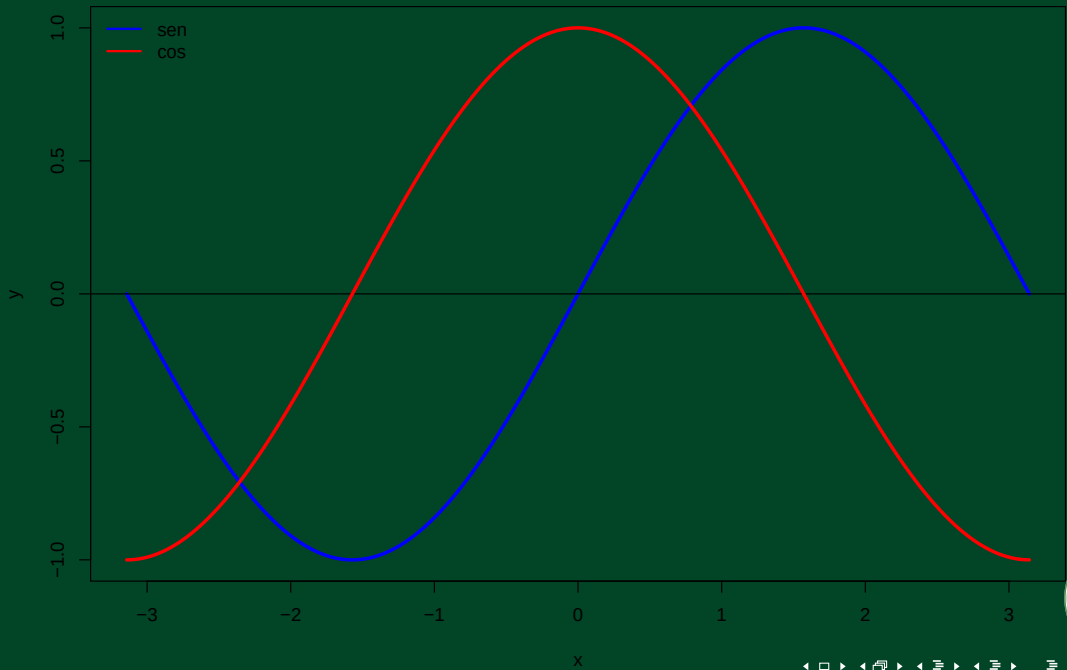


Ejemplo de aplicación

Graficar la función seno (en azul) y coseno (en rojo) , en el mismo sistema de coordenadas y con x variando en el intervalo $(-\pi, \pi)$.

```
x=pretty(c(-pi,pi),5000)
y=sin(x)
plot(x,y,type="l",col="blue",lwd=3)
y1=cos(x)
lines(x,y1,type="l",col="red",lwd=3)
legend("topleft",legend = c("sen", "cos"),lty=1,col=c('blue','red'),bty="n")
```





Mensajes de error



Mensajes de error

```
### not run  
squareroot(4) # la función no existe
```

```
lenght(c(2,4,1))
```

```
length(2)
```

```
[1] 1
```

```
length(c(1,5,6,9))
```

```
[1] 4
```

```
length(2)
```

```
[1] 1
```

```
sqrt 4 # falta el parentesis
```

```
sqrt(4
```



Mensajes de error (II)

```
x
```

```
[1] Inf
```

```
y
```

```
[1] NaN
```

```
sqrt(-2+0i)
```

```
[1] 0+1.414214i
```



Redondeo



Funciones para redondeo

```
# al entero más cercano  
round(127.8976)
```

```
[1] 128
```

```
round(127.4999)
```

```
[1] 127
```

```
# redondeo a tres cifras significativas  
round(127.8976,3)
```

```
[1] 127.898
```

```
## piso  
floor(6.3) # mayor entero que es menor que 6.3
```

```
[1] 6
```

```
floor(-5.8)
```



Funciones para redondeo (II)

```
## Techo
```

```
ceiling(6.3) # menor entero que es mayor que 6.3
```

```
[1] 7
```

```
ceiling(-5.8)
```

```
[1] -5
```



Vectores y asignaciones

R opera sobre estructuras de datos con nombre. La estructura más simple de este tipo es el vector, que es una entidad única que consta de una colección ordenada de datos. Los vectores son la estructura fundamental de datos en R, un número es en realidad un vector de longitud 1. En cuanto al contenido, hay tres tipos principales de vectores, a saber:

- Numéricos (numerical), los cuales pueden ser:
 - De reales (Double), la opción por defecto.
 - De enteros (Integer) para compatibilidad con otros lenguajes.
- De caracteres (Character)
- Lógicos (Logical)

A continuación se crea el objeto (vector) `x` al cual se le asigna el valor de 2,

```
x=2  
mode(x)
```

```
[1] "numeric"
```

```
class(x)
```



Funcion assign()

Otra forma de asignar es con assign() de la siguiente manera.

```
assign("y", c(4,1.2,3,2.5,6) )
```

```
### equivalente a
```

```
y<-c(4,1.2,3,2.5,6)
```

```
### equivalente a
```

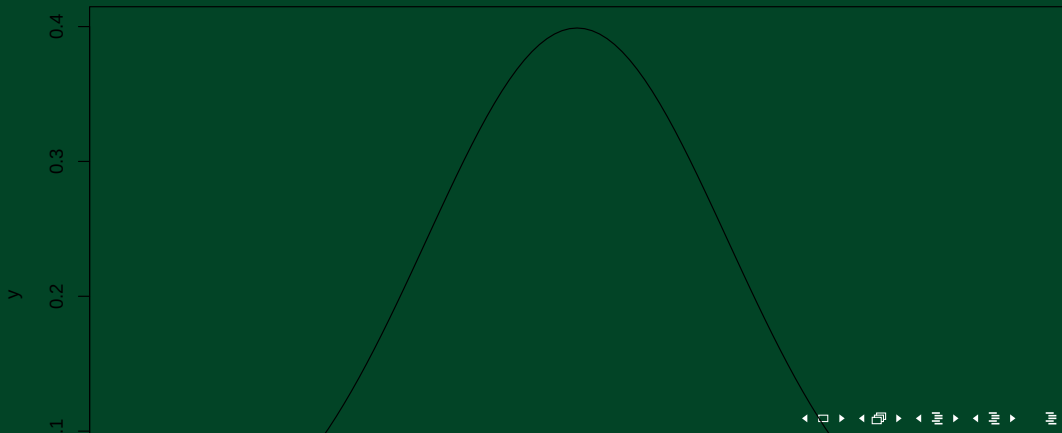
```
y=c(4,1.2,3,2.5,6)
```



Aplicación

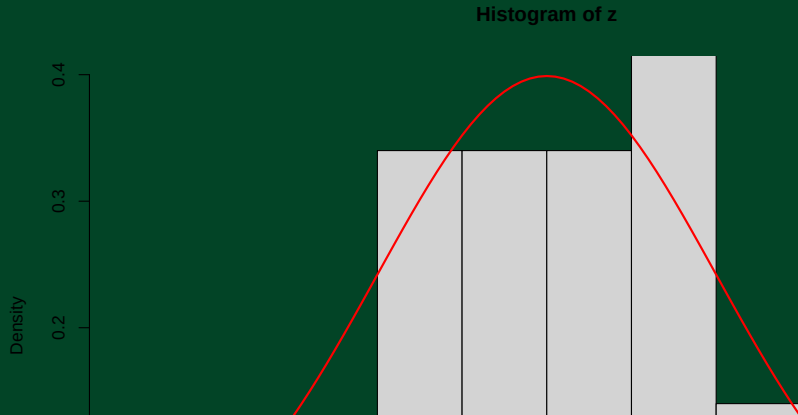
Graficar la función de densidad normal estándar

```
x<-pretty(c(-3,3),100)  
y<-dnorm(x)  
plot(x,y,type="l")
```



Aplicación

```
x<-pretty(c(-3,3),100)
y<-dnorm(x)
z<-rnorm(100)
hist(z,freq=FALSE,ylim=range(y))
lines(x,y, col="red",lwd=2)
```



Caracteres



Vectores de caracteres

¿Qué objetos tengo en memoria?

```
ch<-ls()
```

```
ch
```

```
[1] "x"  "y"  "y1" "z"
```

```
#mode(ch)
```

```
#typeof(ch)
```

```
class(ch)
```

```
[1] "character"
```

```
is.character(ch)
```

```
[1] TRUE
```

El objeto `ch` es un ejemplo de vector de caracteres.

Si el resultado de un calculo no se asigna, este se imprime y se pierde (no se conserva en ningún objeto)



Vectores lógicos

¿Las componentes del vector son iguales a 3?

```
c(TRUE,FALSE,TRUE)
```

```
[1] TRUE FALSE TRUE
```

```
l=c(3, 4 - 1, 1 + 1 + 1,5+2) == 3
```

```
l
```

```
[1] TRUE TRUE TRUE FALSE
```

```
mode(l)
```

```
[1] "logical"
```

```
typeof(l)
```

```
[1] "logical"
```

```
class(l)
```

